

Standard ACLs

- Topics covered:
 - What are ACLs?
 - ACL Logic
 - ACL Types
 - Standard numbered ACLs
 - Standard named ACLs

What Are ACLs?

All About ACLs?

ACLs (Access Control Lists) control access to the network.

They determine which devices are allowed or not allowed to access specific network resources.

Example intent:

- Host A is allowed to access Server A
- Host B is not allowed to access Server A

Previously and later in the course, ACLs are discussed mainly from a security perspective, focusing on controlling which devices can access which parts of the network. Although security is the focus here, ACLs have other uses that are introduced later.

When used for security, ACLs function as a packet filter:

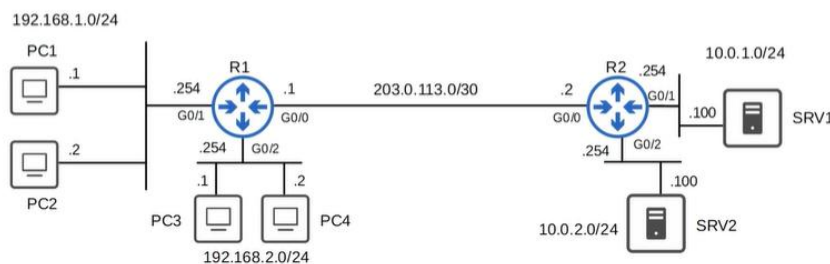
- They instruct the router to permit or discard specific traffic
- Even if a router has a route to a destination, the packet may still be discarded if an ACL says so

ACLs can filter traffic based on:

1. Source IP address
2. Destination IP address
3. Source Layer 4 port
4. Destination Layer 4 port

For CCNA, the focus is on IP addresses and port numbers.

Network Topology:



Notes on representation

- Switches are not shown
- All PCs connect through switches
- This simplified representation is common in network diagrams

How ACLs Work?

We shouldn't just configure ACLs randomly; We must use them to achieve a certain requirement.

Network Policy Requirement

1. Hosts in 192.168.1.0/24:
 - a. Must be able to access 10.0.1.0/24
 - b. Example: access files on SRV1
2. Hosts in 192.168.2.0/24:
 - a. Must not be able to access 10.0.1.0/24
 - b. Example: PC3 and PC4 must not reach SRV1

ACLs must be designed intentionally to meet this requirement.

ACL Basics:

- ACLs are configured in global configuration mode
- An ACL is an ordered list of ACEs (Access Control Entries)

Example of ACL (ACL 1):

1. If source IP = 192.168.1.0/24 → permit
2. If source IP = 192.168.2.0/24 → deny
3. If source IP = any → permit

The order matters explained later.

Creating an ACL does **not** activate it.

It must be applied to an interface:

- Inbound = Interface ke andar aane wala traffic check hota hai
- Outbound = Interface se bahar jaane wala traffic check hota hai

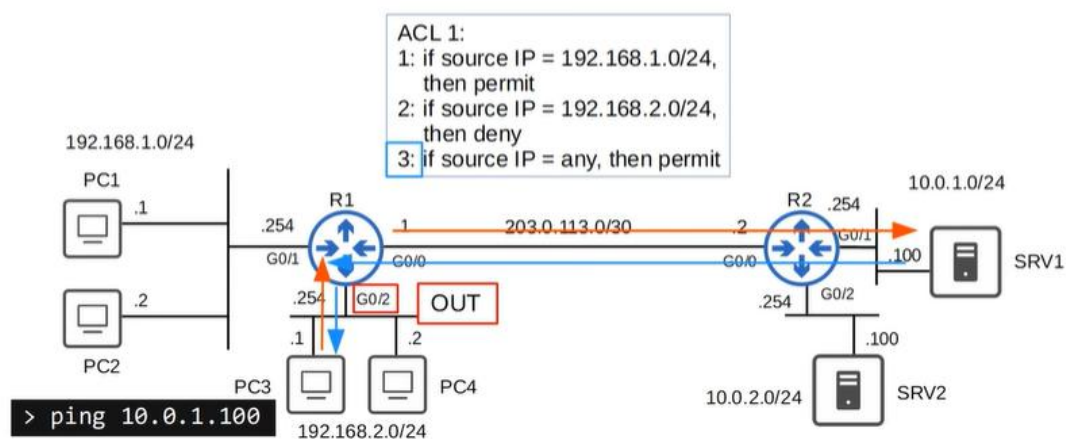
Applying ACL 1

Let see how they work.

Outbound on R1 G0/2 (Incorrect)

Traffic Flow:

PC3 (192.168.2.1) → R1 G0/2 (IN) ---> R1 G0/0 ---> R2 ---> SRV1 → R2 → R1 G0/0 → R1 G0/2 ACL applied Check here



Explanation

1. ACL checks only traffic exiting G0/2
2. Traffic from PC3 enters G0/2 → ACL not checked
3. Packet reaches SRV1
4. Reply traffic:
 - Enters R1 via G0/0
 - Exits G0/2 → ACL checked
 - Source = SRV1
 - Matches “permit any”

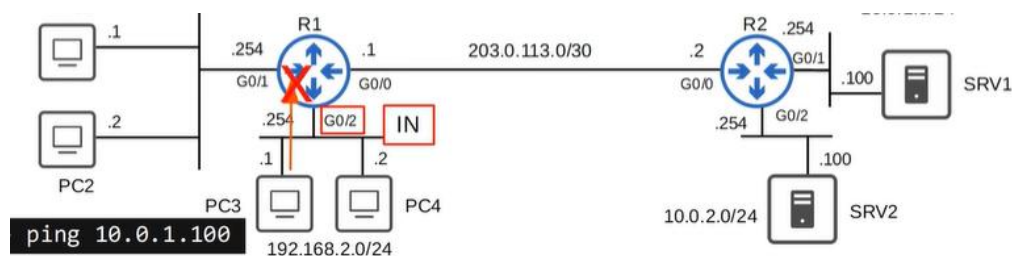
Result:

- PC3 can access SRV1
- Requirement not met

Applying ACL 1 [Inbound on R1 G0/2]

Traffic Flow:

PC3 (192.168.2.1) → R1 G0/2 (IN) X Packet dropped



Explanation

1. Traffic enters G0/2 → ACL checked
2. Entry 1: source ≠ 192.168.1.0/24
3. Entry 2: source ∈ 192.168.2.0/24 → deny
4. Packet dropped
5. Remaining entries ignored

Result:

1. 192.168.2.0/24 blocked from SRV1 (correct)
2. Also blocked from all other networks
3. PCs can only communicate locally

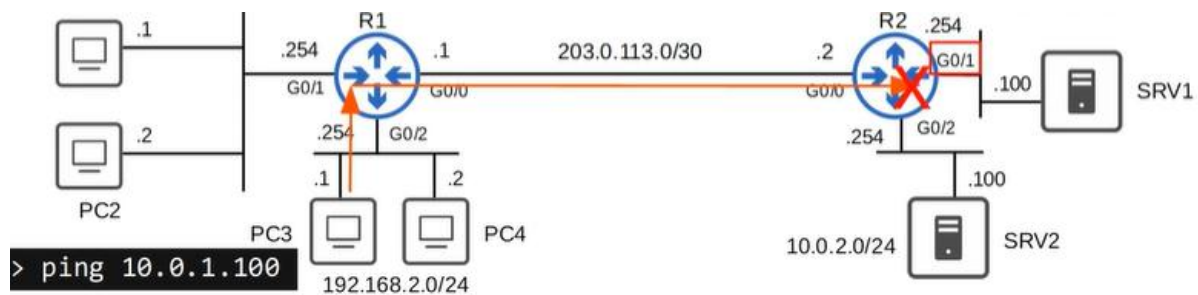
Not the best placement.

Best Placement [Outbound on R2 G0/1]

Traffic from PC3:

PC3 ---> R1 ---> R2 G0/1 (OUT) X SRV1

Source: 192.168.2.0/24 → deny



Traffic from PC1

PC1 ---> R1 ---> R2 G0/1 (OUT) ---> SRV1

Source: 192.168.1.0/24 → permit

Result:

- Both requirements satisfied
- No unintended traffic blocked

ACL Logic

How ACLs Are Processed

1. ACLs are checked top to bottom
2. When a match is found:
 - The specified action is taken
 - Remaining entries are ignored

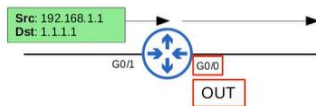
Example: ACL Order Matters

ACL 2 (First Order):

1: permit 192.168.1.0/24

2: deny 192.168.0.0/16

Flow:



Result:

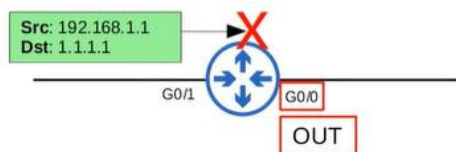
1. Matches (ACL) entry 1
2. Permitted
3. Now ACL Entry 2 ignored

ACL 2 (Reversed Order)

1: deny 192.168.0.0/16

2: permit 192.168.1.0/24

Flow:



Result:

1. Matches entry 1
2. Denied
3. Entry 2 ignored

This shows how critical ACL order is.

One ACL Per Interface Per Direction

1. Maximum per interface:
 - One inbound ACL
 - One outbound ACL
2. Applying a new ACL in the same direction replaces the old one

Implicit Deny

If a packet: Does not match any ACE

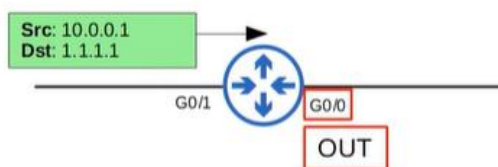
Then: It is denied automatically

This is the implicit deny.

Conceptually, every ACL ends with: deny any

Example ACL:

- 1: permit 192.168.1.0/24
- 2: deny 192.168.0.0/16

Flow:**Result:**

- No match
- Packet denied

Always account for the implicit deny avoiding blocking traffic unintentionally.

ACL Types

Types of ACLs

There are two types of ACLs

1] Standard ACLs

Match based on source IP address only

Subtypes:

1. Standard numbered ACLs
2. Standard named ACLs

2] Extended ACLs

Match based on:

1. Source IP
2. Destination IP
3. Source port
4. Destination port

Subtypes:

1. Extended numbered ACLs
2. Extended named ACLs

All examples shown so far are standard ACLs.

Extended ACLs are covered later.

Standard numbered ACLs

Introduction

Now we move into standard ACLs and how to configure them, starting with standard numbered ACLs.

Standard ACLs:

1. Match traffic only on the source IP address
2. Do not check:
 - Destination IP
 - Source Layer 4 port
 - Destination port
3. Because of this, standard ACLs are simple

Numbered ACLs:

1. Identified using a number
2. Multiple ACLs can exist on one router, so each must be uniquely identified
3. Examples: ACL 1, ACL 2, etc.
4. Named ACLs exist as well, introduced later

Standard ACL Number Ranges

Different ACL types use different number ranges.

For standard ACLs:

- 1–99
- 1300–1999

Originally, only 1–99 was available, limiting standard ACLs to 99 per router.

Later, the range expanded to include 1300–1999, You cannot configure a standard ACL with a number outside these ranges (for example, ACL 100).

In ACL charts, standard ACLs appear as Standard IP ACLs.

IP ACLs are the ACL type required for CCNA.

Protocol	Range
Standard IP	1-99 and 1300-1999
Extended IP	100-199 and 2000-2699
Ethernet type code	200-299
Ethernet address	700-799
Transparent bridging (protocol type)	200-299
Transparent bridging (vendor code)	700-799
Extended transparent bridging	1100-1199
DECnet and extended DECnet	300-399

Xerox Network Systems (XNS)	400-499
Extended XNS	500-599
AppleTalk	600-699
Source-route bridging (protocol type)	200-299
Source-route bridging (vendor code)	700-799
Internetwork Packet Exchange (IPX)	800-899
Extended IPX	900-999
IPX Service Advertising Protocol (SAP)	1000-1099

Basic Command Syntax for Standard Numbered ACLs

The basic command format is:

```
R1(config)# access-list number {deny | permit} ip wildcard-mask
```

- number must be within the standard ACL ranges
- deny or permit specifies the action
- The IP address and wildcard mask define what traffic matches

Wildcard masks are used, not subnet masks.

Configuring a Single Host Entry (/32)

```
1] R1(config)# access-list 1 deny 1.1.1.1 0.0.0.0
```

This denies only host 1.1.1.1 (/32).

You can omit the wildcard mask when matching a single host:

```
2] R1(config)# access-list 1 deny 1.1.1.1
```

Another valid method uses the host keyword.

```
3] R1(config)# access-list 1 deny host 1.1.1.1
```

All three methods are equivalent in effect.

Important:

- The second and third methods work **only for /32**
- For networks like /24, the wildcard mask must be specified

Permitting All Traffic

If the ACL only contains a deny entry, all other traffic will be blocked because of the implicit deny.

CLI Output

```
1] R1(config)# access-list 1 permit any
```

This permits traffic from any source IP.

Equivalent command using IP and wildcard mask:

```
2] R1(config)# access-list 1 permit 0.0.0.0 255.255.255.255
```

Both commands match all IP addresses.

Adding a Remark:

Remarks are descriptive comments:

- They do not affect ACL behaviour
- They help explain the ACL's purpose

CLI Output

```
R1(config)# access-list 1 remark ## BLOCK BOB FROM ACCOUNTING ##
```

The hash symbols are optional and used only for readability.

Viewing the ACL Configuration

```
R1(config)#access-list 1 deny 1.1.1.1 0.0.0.0
R1(config)#access-list 1 permit 0.0.0.0 255.255.255.255
R1(config)#access-list 1 remark ## BLOCK BOB FROM ACCOUNTING ##
```

CLI Output:

```
R1(config)#do show access-lists
Standard IP access list 1
  10 deny 1.1.1.1
  20 permit any
R1(config)#
```

Key points:

1. /32 entries are displayed without wildcard masks
2. 0.0.0.0 255.255.255.255 is shown as any
3. Remarks are not shown here
4. Sequence numbers (10, 20) indicate processing order

CLI Output:

```
R1(config)#do show ip access-lists
Standard IP access list 1
  10 deny 1.1.1.1
  20 permit any
R1(config)#
```

This output is identical but shows **only IP ACLs**.

CLI Output:

```
R1(config)#do show running-config | include access-list
access-list 1 deny 1.1.1.1
access-list 1 permit any
access-list 1 remark ## BLOCK BOB FROM ACCOUNTING ##
R1(config)#
```

Here:

1. Remarks are visible
2. Entries appear exactly as stored in the configuration

Applying an ACL to an Interface

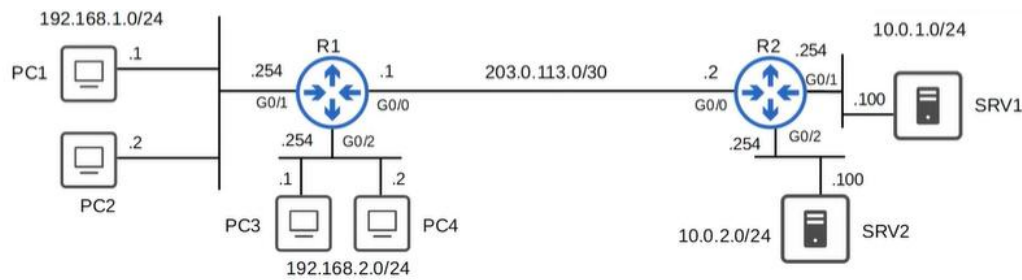
ACLs do not take effect until applied to an interface.

Command Syntax:

```
R1(config-if)# ip access-group number {in | out}
```

This command is entered in interface configuration mode.

Real Configuration Example



Requirements:

1. PC1 (192.168.1.1) can access 192.168.2.0/24
2. Other PCs in 192.168.1.0/24 cannot access 192.168.2.0/24

ACL Configuration to Meet Requirements

CLI Output:

```
R1(config)# access-list 1 permit 192.168.1.1
```

```
R1(config)# access-list 1 deny 192.168.1.0 0.0.0.255
```

```
R1(config)# access-list 1 permit any
```

Explanation:

1. First entry permits PC1 only
2. Second entry denies the rest of the subnet
3. Final permit avoids blocking all other traffic due to implicit deny

Applying the ACL

CLI Output:

```
R1(config)# interface g0/2
```

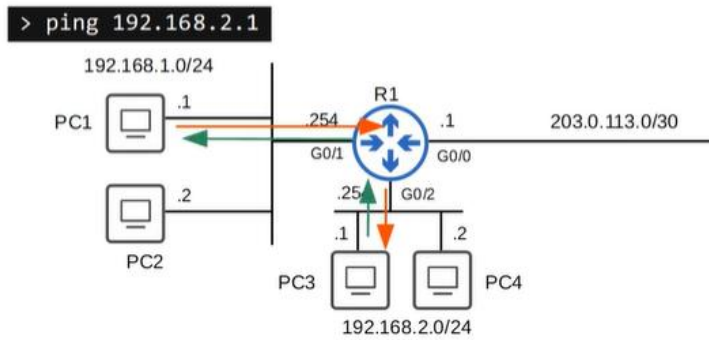
```
R1(config-if)# ip access-group 1 out
```

Rule of thumb for standard ACLs:

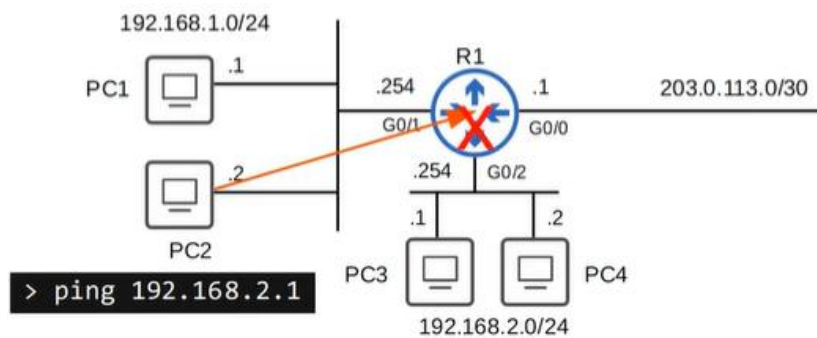
- Apply them as close to the destination as possible

Here, the destination is 192.168.2.0/24, so outbound on G0/2 is correct.

Traffic Flow — PC1 (Permitted):



Traffic Flow — PC2 (Denied):



Key Takeaways from This Section

1. Standard ACLs match only source IP
2. ACL number ranges matter & Order of entries is critical
3. Implicit deny exists at the end, ACLs must be applied to an interface
4. Standard ACLs should be placed as close to the destination as possible

This completes the standard numbered ACL configuration walkthrough.

Standard Named ACLs

Introduction

Standard named ACLs are still standard ACLs, so they, Match traffic only by source IP address

The difference:

- They are identified by a name, not a number
- Example name: BLOCK_BOB

Entering Standard Named ACL Configuration Mode:

Standard named ACLs are configured by entering standard named ACL config mode, then configuring each entry inside that mode.

Command to enter config mode:

```
R1(config)# ip access-list standard acl-name
```

Important:

- The command uses IP ACCESS-LIST, not ACCESS-LIST
- This differs from standard numbered ACLs

Configuring Entries Inside Named ACL Mode:

```
R1(config-std-nacl)# [entry-number] {deny | permit} ip wildcard-mask
```

Once inside standard named ACL config mode.

1. You configure deny and permit entries
2. You can specify entry numbers, but you don't have to
3. If you don't specify numbers:
 - Entries are automatically numbered 10, 20, 30, etc.
4. Manually specifying numbers allows precise control of entry order

Example: Creating a Standard Named ACL

```
R1(config)#ip access-list standard BLOCK_BOB
R1(config-std-nacl)#5 deny 1.1.1.1
R1(config-std-nacl)#10 permit any
R1(config-std-nacl)#remark ## CONFIGURED NOV 21 2020 ##
```

Explanation:

1. ACL named BLOCK_BOB is created
2. Entry number 5 is manually specified
3. Entry number 10 is manually specified
4. A remark is added for documentation

Applying the Named ACL to an Interface

```
R1(config-std-nacl)#interface g0/0
R1(config-if)#ip access-group BLOCK_BOB in
```

The ACL is applied the same way as numbered ACLs:

1. ip access-group
2. ACL name
3. in or out

Verifying the Named ACL

```
R1#show access-lists
Standard IP access list BLOCK_BOB
 5 deny 1.1.1.1
10 permit any
```

1. Entry numbers appear
2. ACL name is displayed

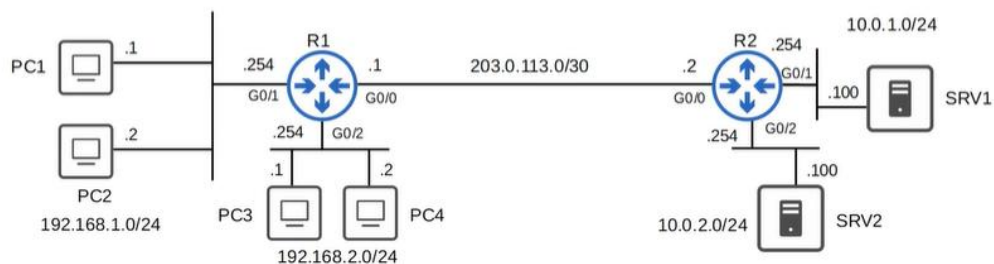
Viewing the ACL in the Running Configuration

```
R1#show running-config | section access-list
ip access-list standard BLOCK_BOB
 deny 1.1.1.1
 permit any
 remark ## CONFIGURED NOV 21 2020 ##
```


Key points:

1. SECTION ACCESS-LIST shows the full ACL
2. INCLUDE ACCESS-LIST would not show the entries
3. Entry numbers are not displayed in the config
4. The remark is visible

Named ACL Configuration on R2



Requirements:

1. PCs in 192.168.1.0/24 can't access 10.0.2.0/24
2. PC3 can't access 10.0.1.0/24
3. Other PCs in 192.168.2.0/24 can access 10.0.1.0/24
4. PC1 can access 10.0.1.0/24
5. Other PCs in 192.168.1.0/24 can't access 10.0.1.0/24

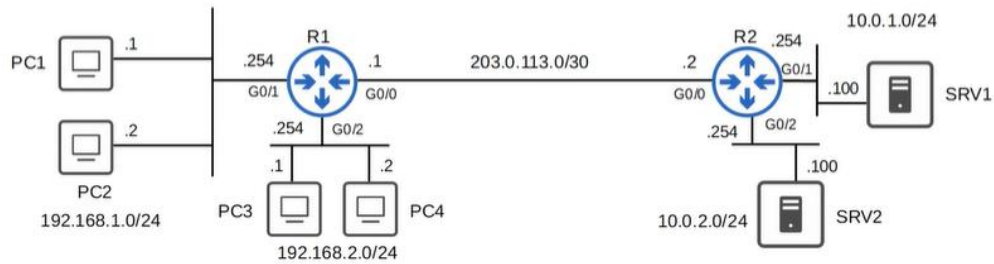
Two ACLs are required.

ACL 1 — Controlling Access to 10.0.2.0/24

```
R2(config)#ip access-list standard TO_10.0.2.0/24
R2(config-std-nacl)#deny 192.168.1.0 0.0.0.255
R2(config-std-nacl)#permit any
R2(config-std-nacl)#interface g0/2
R2(config-if)#ip access-group TO_10.0.2.0/24 out
```

Effect:

1. PC1 and PC2 are blocked from SRV2
2. PC3 and PC4 are permitted



ACL 2 — Controlling Access to 10.0.1.0/24

```
R2(config-if)#ip access-list standard TO_10.0.1.0/24
R2(config-std-nacl)#deny 192.168.2.1
R2(config-std-nacl)#permit 192.168.2.0 0.0.0.255
R2(config-std-nacl)#permit 192.168.1.1
R2(config-std-nacl)#deny 192.168.1.0 0.0.0.255
R2(config-std-nacl)#permit any
R2(config-std-nacl)#interface g0/1
R2(config-if)#ip access-group TO_10.0.1.0/24 out
```

Verifying the ACLs

```
R2#show ip access-lists
Standard IP access list TO_10.0.1.0/24
 30 permit 192.168.1.1
 10 deny 192.168.2.1
 20 permit 192.168.2.0, wildcard bits 0.0.0.255
 40 deny 192.168.1.0, wildcard bits 0.0.0.255
 50 permit any
Standard IP access list TO_10.0.2.0/24
 10 deny 192.168.1.0, wildcard bits 0.0.0.255
 20 permit any
```

ACL Entry Reordering Behaviour

Observation:

1. The sequence numbers reflect configuration order
2. The actual ACL order is different

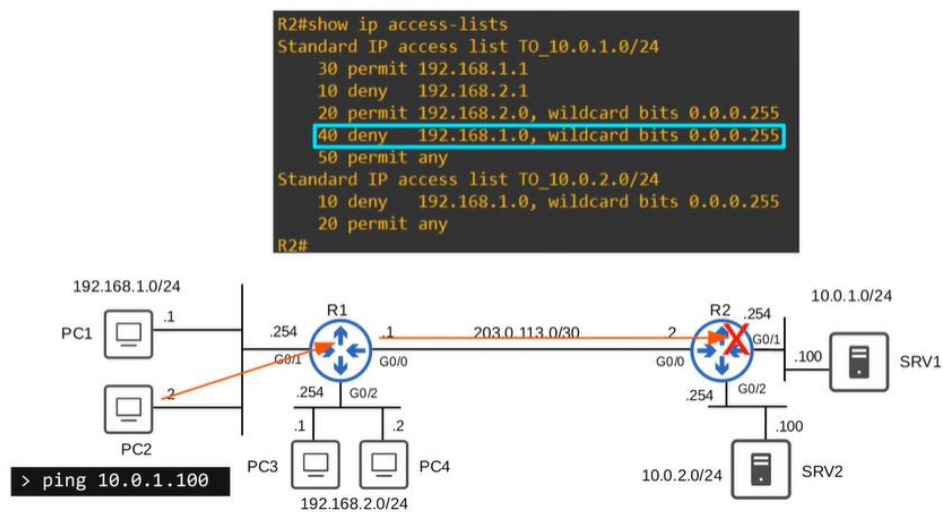
Reason:

1. The router may re-order /32 host entries
2. This improves processing efficiency
3. The overall ACL behaviour does not change

Notes: This can occur with both numbered and named ACLs & Packet Tracer does **not** reorder entries

Final Traffic Example — PC2 to SRV1

Traffic Flow:



ACL processing:

- Does not match first entries
- Matches entry:
 - deny 192.168.1.0 0.0.0.255
- Packet is denied

Result:

- PC2 cannot access SRV1

This completes the walkthrough of standard named ACLs, their configuration, verification, and behaviour.